This completes our development of the allocation model. Even though we have not developed it entirely, the precision in some of the terms indicates how one goes about formulating such a problem. In addition to this aspect, we have indicated the important issues that need to be addressed in allocation models.

2.2.3 Solution Methods

As noted earlier, simple file allocation problem is NP-complete. Since the model we developed in the previous section is more complex, it is likely to be NP-complete as well. Thus one has to look for heuristic methods that yield suboptimal solutions. The test of "goodness" in this case is, obviously, how close the results of the heuristic algorithm are to the optimal allocation.

It was observed early on that there is a correspondence between the file allocation and the facility location problems. In fact, the isomorphism of the simple file allocation problem and the single commodity warehouse location problem has been shown. Thus, heuristics developed for the latter have been used for the former. Examples are the knapsack problem solution, branch-and-bound techniques, and network flow algorithms.

There have been other attempts to reduce the complexity of the problem. One strategy has been to assume that all the candidate partitionings have been determined together with their associated costs and benefits in terms of query processing. The problem, then, is modeled as choosing the optimal partitioning and placement for each relation. Another simplification frequently employed is to ignore replication at first and find an optimal nonreplicated solution. Replication is handled at the second step by applying a greedy algorithm which starts with the nonreplicated solution as the initial feasible solution, and tries to improve upon it. For these heuristics, however, there is not enough data to determine how close the results are to the optimal.

2.3 Combined Approaches

The design process depicted in Fig. 2.1 on which we based our discussion separates the fragmentation and allocation steps. The methodology is linear where the output of fragmentation is input to allocation; we call this the *fragment-then-allocate approach*. This simplifies the formulation of the problem by reducing the decision space, but the isolation of the two steps may in fact contribute to the complexity of the allocation models. Both steps have similar inputs, differing only in that fragmentation works on global relations, whereas allocation considers fragments. They both require workload information, but ignore how each other makes use of these inputs. The end result is that the fragmentation algorithms decide how to partition a relation based partially on how queries access it, but the allocation models

ignore the part that this input plays in fragmentation. Therefore, the allocation models have to include all over again detailed specification of the relationship among the fragment relations and how user applications access them. There are approaches that combine the fragmentation and allocation steps in such a way that the data partitioning algorithm also dictates allocation, or the allocation algorithm dictates how the data is partitioned; we call these the *combined approaches*. These mostly consider horizontal partitioning, since that is the common method for obtaining significant parallelism. In this section we present these approaches, classified as either workload-agnostic or workload-aware.

2.3.1 Workload-Agnostic Partitioning Techniques

This class of techniques ignores the workload that will run on the data and simply focus on the database, often not even paying attention to the schema definition. These approaches are mostly used in parallel DBMSs where data dynamism is higher than distributed DBMSs, so simpler techniques that can be quickly applied are preferred.

The simplest form of these algorithms is *round-robin partitioning* (Fig. 2.19). With n partitions, the ith tuple in insertion order is assigned to partition (i mod n). This strategy enables the sequential access to a relation to be done in parallel. However, the direct access to individual tuples, based on a predicate, requires accessing the entire relation. Thus, round-robin partitioning is appropriate for full scan queries, as in data mining.

An alternative is *hash partitioning*, which applies a hash function to some attribute that yields the partition number (Fig. 2.20). This strategy allows exact-match queries on the selection attribute to be processed by exactly one node and all

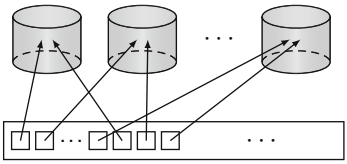

Fig. 2.19 Round-robin partitioning

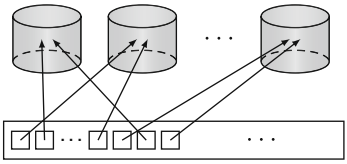

Fig. 2.20 Hash partitioning

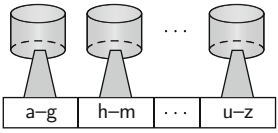

Fig. 2.21 Range partitioning

other queries to be processed by all the nodes in parallel. However, if the attribute used for partitioning has nonuniform data distribution, e.g., as with people's names, the resulting placement may be unbalanced, with some partitions much bigger than some others. This is called *data skew* and it is an important issue that can cause unbalanced load.

Finally, there is *range partitioning* (Fig. 2.21) that distributes tuples based on the value intervals (ranges) of some attribute and thus can deal with nonuniform data distributions. Unlike hashing, which relies on hash functions, ranges must be maintained in an index structure, e.g., a B-tree. In addition to supporting exact-match queries (as in hashing), it is well-suited for range queries. For instance, a query with a predicate "A between A_1 and A_2" may be processed by the only node(s) containing tuples whose A value is in range $[A_1, A_2]$.

These techniques are simple, can be computed quickly and, as we discuss in Chap. 8, nicely fit the dynamicity of data in parallel DBMSs. However, they have indirect ways of handling the semantic relationships among relations in the database. For example, consider two relations that have a foreign key–primary key join relationship such as $\texttt{R} \bowtie_{\texttt{R.A=S.B}} \texttt{S}$, hash partitioning would use the same function over attribute `R.A` and `S.B` to ensure that they are located at the same node, thereby localizing the joins and parallelizing the join execution. A similar approach can be used in range partitioning, but round-robin would not take this relationship into account.

2.3.2 Workload-Aware Partitioning Techniques

This class of techniques considers the workload as input and performs partitioning to localize as much of the workload on one site as possible. As noted at the beginning of this chapter, their objective is to minimize the amount of distributed queries.

One approach that has been proposed in a system called Schism uses the database and workload information to build a graph $G = V, E$ where each vertex v in V represents a tuple in the database, and each edge $e = (v_i, v_j)$ in E represents a query that accesses both tuples v_i and v_j. Each edge is assigned a weight that is the count of the number of transactions that access both tuples.

In this model, it is also easy to take into account replicas, by representing each copy by a separate vertex. The number of replica vertices is determined by the number of transactions accessing the tuple; i.e., each transaction accesses one

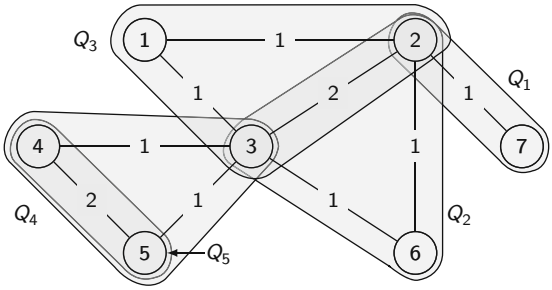

Fig. 2.22 Graph representation for partitioning in schism

copy. A replicated tuple is represented in the graph by a star-shaped configuration consisting of $n+1$ vertices where the "central" vertex represents the logical tuple and the other n vertices represent the physical copies. The weight of an edge between the physical copy vertex and the central vertex is the number of transactions that update the tuple; the weights of other edges remain as the number of queries that access the tuple. This arrangement makes sense since the objective is to localize transactions as much as possible and this technique uses replication to achieve localization.

Example 2.19 Let us consider a database with one relation consisting of seven tuples that are accessed by five transactions. In Fig. 2.22 we depict the graph that is constructed: there are seven vertices corresponding to the tuples, and the queries that access them together are shown as cliques. For example, query Q_1 accesses tuples 2 and 7, query Q_2 accesses tuples 2, 3, and 6, query Q_3 accesses tuples 1, 2, and 3, query Q_4 accesses tuples 3, 4, and 5, and query Q_5 accesses tuples 4 and 5. Edge weights capture the number of transaction accesses.

Replication can be incorporated into this graph but replicating the tuples that are accessed by multiple transactions; this is shown in Fig. 2.23. Note that tuples 1, 6, and 7 are not replicated since they are only accessed by one transaction each, tuples 4 and 5 are replicated twice, and tuples 2 and 3 are replicated three times. We represent the "replication edges" between the central vertex and each physical copy by dashed lines and omit the weights for these edges in this example. ♦

Once the database and the workload are captured by this graph representation, the next step is to perform a vertex-disjoint graph partitioning. Since we discuss these techniques in detail in Sect. 10.4.1, we do not get into the details here, but simply state that vertex-disjoint partitioning allocates each vertex of the graph to a separate partition such that partitions are mutually exclusive. These algorithms have, as their objective function, a balanced (or nearly balanced) set of partitions while minimizing the cost of edge cuts. The cost of an edge cut takes into account the weights of each edge so as to minimize the number of distributed queries.

The advantage of the Schism approach is its fine-grained allocation—it treats each tuple as an allocation unit and the partitioning "emerges" as the allocation decision is made for each tuple. Thus, the mapping of sets of tuples to queries can

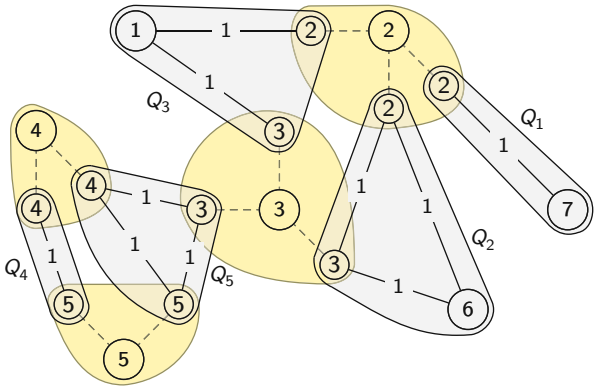

Fig. 2.23 Schism graph incorporating replication

be controlled and many of them can execute at one site. However, the downside of the approach is that the graph becomes very large as the database size increases, in particular when replicas are added to the graph. This makes the management of the graph difficult and its partitioning expensive. Another issue to consider is that the mapping tables that record where each tuple is stored (i.e., the directory) become very large and may pose a management problem of their own.

One approach to overcome these issues has been proposed as part of the SWORD system that employs a hypergraph model[7] where each clique in Fig. 2.22 is represented as a hyperedge. Each hyperedge represents one query and the set of vertices spanned by the hyperedge represents the tuples accessed by it. Each hyperedge has a weight that represents the frequency of that query in the workload. Therefore what we have is a weighted hypergraph. This hypergraph is then partitioned using a k-way balanced min-cut partitioning algorithm that produces k balanced partitions, each of which is allocated to a site. This minimizes the number of distributed queries since the algorithm is minimizing the cuts in hyperedges and each of these cuts indicates a distributed query.

Of course, this change in the model is not sufficient to address the issues discussed above. In order to reduce the size of the graph, and the overhead of maintaining the associated mapping table, SWORD compresses this hypergraph as follows. The set of vertices V in the original hypergraph G is mapped to a set of virtual vertices V' using a hash or other function that operates on the primary keys of the tuples. Once the set of virtual vertices are determined, the edges in the original hypergraph are now mapped to hyperedges in the compressed graph (E') such that if the vertices spanned by a hyperedge $e \in E$ are mapped to different virtual vertices in the compressed graph, then there will be a hyperedge $e' \in E'$.

[7] A hypergraph allows each edge (called a hyperedge) to connect more than two vertices as is the case with regular graphs. The details of the hypergraph model are beyond our scope.

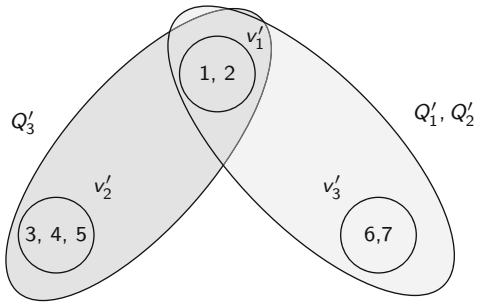

Fig. 2.24 Sword compressed hypergraph

Of course for this compression to make sense, $|V'| < |V|$, so a critical issue is to determine how much compression is desired—too much compression will reduce the number of virtual vertices, but will increase the number of hyperedges, and therefore the possibility of distributed queries. The resulting compressed hypergraph $G' = (V', E')$ is going to be smaller than the original hypergraph so easier to manage and partition, and the mapping tables will also be smaller since they will only consider the mapping of sets of virtual vertices.

Example 2.20 Let us revisit the case in Example 2.19 and consider that we are compressing the hypergraph into three virtual vertices: $v'_1 = 1, 2$, $v'_2 = 3, 4, 5$, $v'_3 = 6, 7$. Then there would be two hyperedges: $e'_1 = (v'_1, v'_3)$ with frequency 2 (corresponding to Q_1 and Q_2 in the original hypergraph) and $e'_2 = (v'_1, v'_2)$ with frequency 1 (corresponding to Q_3). The hyperedges representing queries Q_4 and Q_5 would be local (i.e., not spanning virtual vertices) so no hyperedges are required in the compressed hypergraph. This is shown in Fig. 2.24. ♦

Performing the k-way balanced min-cut partitioning on the compressed hypergraph can be performed much faster and the resulting mapping table will be smaller due to the reduced size of the graph.

SWORD incorporates replication in the compressed hypergraph. It first determines, for each virtual vertex, how many replicas are required. It does this by using the tuple-level access pattern statistics for each tuple t_j in each virtual vertex v'_i, namely its read frequency f^r_{ij} and its write frequency f^w_{ij}. Using these, it computes the average read and write frequencies (ARF and AWF, respectively) of virtual vertex v'_i as follows:

$$ARF(v'_i) = \frac{\sum_j f^r_{ij}}{\log S(v'_i)} \quad \text{and} \quad AWF(v'_i) = \frac{\sum_j f^w_{ij}}{\log S(v'_i)}$$

$S(v'_i)$ is the size of each virtual vertex (in terms of the number of actual vertices mapped to it) and its log is taken to compensate for the skew in the sizes of virtual vertices (so, these are size-compensated averages). From these, SWORD defines a

replication factor, $\mathcal{R} = \frac{AWF(v_i')}{ARWF(v_i')}$ and a user-specified threshold δ $(0 < \delta < 1)$ is defined. The number of replicas (#_rep) for virtual vertex v_i' is then given as

$$\#_rep(v_i') = \begin{cases} 1 & \text{if } \mathcal{R} \geq \delta \\ ARF(v_i') & \text{otherwise} \end{cases}$$

Once the number of replicas for each virtual vertex is determined, these are added to the compressed hypergraph and assigned to hyperedges in a way that minimizes the min-cut in the partitioning algorithm. We ignore the details of this assignment.

2.4 Adaptive Approaches

The work described in this chapter generally assumes a static environment where design is conducted only once and this design can persist. Reality, of course, is quite different. Both physical (e.g., network characteristics, available storage at various sites) and logical (e.g., workload) changes occur necessitating redesign of the database. In a dynamic environment, the process becomes one of design-redesign-materialization of the redesign. When things change, the simplest approach is to redo the distribution design from scratch. For large or highly dynamic systems, this is not quite realistic as the overhead of redesign is likely to be very high. A preferred approach is to perform incremental redesign, focusing only on the parts of the database that are likely to be affected by the changes. The incremental redesign can either be done every time a change is detected or periodically where changes are batched and evaluated at regular intervals.

Most of the work in this area has focused on changes in the workload (queries and transactions) over time and those are what we focus in this section. While some work in this area has focused on the fragment-then-allocate approach, most follow the combined approach. In the former case one alternative that has been proposed involves a split phase where fragments are further subdivided based on the changed application requirements until no further subdivision is profitable based on a cost function. At this point, the merging phase starts where fragments that are accessed together by a set of applications are merged into one fragment. We will focus more on the dynamic combined approaches that perform incremental redesign as the workload changes.

The objective in the adaptive approaches is the same as the workload-aware partitioning strategies discussed in Sect. 2.3.2: to minimize the number of distributed queries and ensure that the data for each query is local. Within this context, there are three interrelated issues that need to be addressed in adaptive distribution design:

1. How to detect workload changes that require changes in the distribution design?
2. How to determine which data items are going to be affected in the design?
3. How to perform the changes in an efficient manner?

In the remainder we discuss each of these issues.

2.4.1 Detecting Workload Changes

This is a difficult issue on which there is not much work. Most of the adaptive techniques that have been proposed assume that the change in the workload is detected, and simply focus on the migration problem. To be able to detect workload changes, the incoming queries need to be monitored. One way to do this is to periodically examine the system logs, but this may have high overhead, especially in highly dynamic systems. An alternative is to continuously monitor the workload within the DBMS. In the SWORD system we discussed above, the system monitors the percentage increase in the number of distributed transactions and considers that the system has changed sufficiently to require a reconfiguration if this percentage increase is above a defined threshold. As another example, the E-Store system monitors both system-level metrics and tuple-level access. It starts by collecting system-level metrics at each computing node using OS facilities. E-Store currently focuses primarily on detecting workload imbalances across the computing nodes, and therefore only collects CPU utilization data. If the CPU utilization imbalance exceeds a threshold, then it invokes more fine-grained tuple-level monitoring to detect the affected items (see next section). Although imbalance in CPU utilization may be a good indicator of possible performance problems, it is too simple to capture more significant workload changes. It is possible, of course, to do more sophisticated monitoring, e.g., one can create a profile that looks at the frequency of each query in a given time period, the percentage of queries that meet (or exceed) their agreed-upon latencies (as captured in a service level agreement, perhaps), and others. Then it can be decided whether the changes in the profile require redesign, which can be done either continuously (i.e., every time the monitor registers information) or periodically. The challenge here is to do this efficiently without intruding on the system performance. This is an open research area that has not been properly studied.

2.4.2 Detecting Affected Items

Once a change is detected in the workload the next step is to determine what data items are affected and need to be migrated to address this change. How this is done is very much dependent on the detection method. For example, if the system is monitoring the frequency of queries and detects changes, then the queries will identify the data items. It is possible to generalize from individual queries to query templates in order to capture "similar" queries that might also be affected by the changes. This is done in the Apollo system where each constant is replaced by a wildcard. For example, the query

```
SELECT PNAME FROM PROJ WHERE BUDGET>200000 AND LOC = "London"
```

would be generalized to

```
SELECT PNAME FROM PROJ WHERE BUDGET>? AND LOC = "?"
```

While this reduces the granularity of determining the exact set of data items that are affected, it may allow the detection of additional data items that might be affected by similar queries and reduce the frequency of changes that are necessary.

The E-Store system starts tuple-level monitoring once it detects a system load imbalance. For a short period, it collects access data to the tuples in each computing node (i.e., each partition) and determines the "hot" tuples, which are the top-k most frequently accessed tuples within a time period. To do this, it uses a histogram for each tuple that is initialized when the tuple-level monitoring is enabled and updated as access happens within the monitoring window. At the end of this time period, the top-k list is assembled. The monitoring software gathers these lists and generates a global top-k list of hot tuples—these are the data items that need to be migrated. A side-effect is the determination of cold tuples; of particular importance are tuples that were previously hot and have since become cold. The determination of the time window for tuple-level monitoring and the value of k are parameters set by the database administrator.

2.4.3 Incremental Reconfiguration

As noted earlier, the naive approach to perform redesign is to redo the entire data partitioning and distribution. While this may be of interest in environments where workload change occurs infrequently, in most cases, the overhead of redesign is too high to do it from scratch. The preferred approach is to apply the changes incrementally by migrating data; in other words, we only look at the changed workload and the data items that are affected, and move them around.[8] So, in this section, we focus on incremental approaches.

Following from the previous section, one obvious approach is to use an incremental graph partitioning algorithm that reacts to changes in the graph representation we discussed. This has been followed in the SWORD system discussed above and in AdaptCache, both of which represent usage as hypergraphs and perform incremental partitioning on these graphs. The incremental graph partitioning initiates data migration for reconfiguration.

The E-Store system we have been discussing takes a more sophisticated approach. Once the set of hot tuples are identified, a migration plan is prepared that identifies where the hot tuples should be moved and what reallocation of cold tuples is necessary. This can be posed as an optimization problem that creates a balanced load across the computing nodes (balance is defined as average-load-across-nodes

[8]The research in this area has exclusively focused on horizontal partitioning, which will be our focus here as well, meaning that our units of migration are individual tuples.

± a threshold value), but solving this optimization problem in real time for online reconfiguration is not easy, so it uses approximate placement approaches (e.g., greedy, first-fit) to generate the reconfiguration plan. Basically, it first determines the appropriate computing nodes at which each hot tuple should be located, then addresses cold tuples, if necessary due to remaining imbalance, by moving them in blocks. So, the generated reconfiguration plan addresses the migration of hot tuples individually, but the migration of cold tuples as blocks. As part of the plan a coordinating node is determined to manage the migration, and this plan is an input to the Squall reconfiguration system.

Squall performs reconfiguration and data migration in three steps. In the first step, the coordinator identified in the reconfiguration plan initializes the system for migration. This step includes the coordinating obtaining exclusive access control to all of these partitions through a transaction as we will discuss in Chap. 5. Then the coordinator asks each site to identify the tuples that will be moving out of the local partition and the tuples that will be coming in. This analysis is done on the metadata so can be done quickly after which each site notifies the coordinator and the initialization transaction terminates. In the second step, the coordinator instructs each site to do the data migration. This is critical as there are queries accessing the data as it is being moved. If a query is executing at a given computing node where the data is supposed to be according to the reconfiguration plan but the required tuples are not locally available, Squall pulls the missing tuples to process the query. This is done in addition to the normal migration of the data according to the reconfiguration plan. In other words, in order to execute the queries in a timely fashion, Squall performs on-demand movement in addition to its normal migration. Once this step is completed, each node informs the coordinator, which then starts the final termination step and informs each node that reconfiguration is completed. These three steps are necessary for Squall to be able to perform migration while executing user queries at the same time rather than stopping all query execution, performing the migration and then restarting the query execution.

Another approach is *database cracking*, which is an adaptive indexing technique that targets dynamic, hard to predict workloads and scenarios where there is little or no idle time to devote to workload analysis and index building. Database cracking works by continuously reorganizing data to match the query workload. Every query is used as an advice on how the data should be stored. Cracking does this by building and refining indices partially and incrementally as part of query processing. By reacting to every single query with lightweight actions, database cracking manages to adapt to a changing workload instantly. As more queries arrive, the indices are refined, and the performance improves, eventually reaching the optimal performance, i.e., the performance we would get from a manually tuned system.

The main idea in the original database cracking approach is that the data system reorganizes one column of the data at a time and only when touched by a query. In other words, the reorganization utilizes the fact that the data is already read and decides how to refine it in the best way. Effectively the original cracking approach overloads the select operator of a database system and uses the predicates of each query to determine how to reorganize the relevant column. The first time an attribute

A is required by a query, a copy of the base column *A* is created, called the cracker column of *A*. Each select operator on *A* triggers the physical reorganization of the cracker column based on the requested range of the query. Entries with a key that is smaller than the lower bound are moved before the lower bound, while entries with a key that is greater than the upper bound are moved after the upper bound in the respective column. The partitioning information for each cracker column is maintained in an AVL-tree, the cracker index. Future queries on column *A* search the cracker index for the partition where the requested range falls. If the requested key already exists in the index, i.e., if past queries have cracked on exactly those ranges, then the select operator can return the result immediately. Otherwise, the select operator refines on the fly the column further, i.e., only the partitions/pieces of the column where the predicates fall will be reorganized (at most two partitions at the boundaries of the range). Progressively the column gets more "ordered" with more but smaller pieces.

The primary concept in database cracking and its basic techniques can be extended to partition data in a distributed setting, i.e., to store data across a set of nodes using incoming queries as an advice. Each time a node needs a specific part of the data for a local query but the data does not exist in this node, this information can be used as a hint that the data could be moved to this node. However, contrary to the in-memory database cracking methods where the system reacts immediately to every query, in a distributed setting we need to consider that moving the data is more expensive. At the same time, for the same reason, the benefit that future queries may have is going to be more significant. In fact, the same trade-off has already been studied in variations of the original database cracking approach to optimize for disk-based data. The net effect is twofold: (1) instead of reacting with every query, we should wait for more workload evidence before we embark on expensive data reorganization actions, and (2) we should apply "heavier" reorganizations to utilize the fact that reading and writing data is more expensive out of memory. We expect future approaches to explore and develop such adaptive indexing methods to benefit from effective partitioning in scenarios where the workload is not easy to predict, and there is not enough time to fully sort/partition all data before the first query arrives.

2.5 Data Directory

The final distribution design issue we discuss is related to data directory. The distributed database schema needs to be stored and maintained by the system. This information is necessary during distributed query optimization, as we will discuss later. The schema information is stored in a *catalog/data dictionary/directory* (simply directory). A directory is a metadatabase that stores a number of information such as schema and mapping definitions, usage statistics, access control information, and the like.